SOFTEX PERNAMBUCO

Softex

MINISTÉRIO DA CIÊNCIA, TECNOLOGIA E INOVAÇÃO

GOVERNO FEDERAL BRASIL UNIÃO E RECONSTRUÇÃO

<\>

Curso de programação Back-end

JavaScript- Node.js - Express.js

BOLSA FUTURO DIGITAL

SOFTEX PERNAMBUCO

Softex

MINISTÉRIO DA CIÊNCIA, TECNOLOGIA E INOVAÇÃO

GOVERNO FEDERAL BRASIL UNIÃO E RECONSTRUÇÃO

Git, Github, Gitlab

Antes de mais nada, um panorama geral. Se você é um desenvolvedor ou está considerando entrar na área de tecnologia, é provável que já tenha se deparado com os termos "Git" e "GitHub". À primeira vista, pode parecer que "Git" e "GitHub" são a mesma coisa, mas a resposta é não. Embora sejam ferramentas distintas, elas colaboram de forma integrada para tornar o desenvolvimento de software mais eficiente. A seguir, apresentamos um resumo rápido de cada uma delas, o que também ajuda a diferenciá-las:

- Git: É um sistema de controle de versão de arquivos. Em outras palavras, é a ferramenta responsável por registrar o histórico de alterações sempre que um arquivo monitorado por ele é modificado.
- GitHub: É uma "rede social para desenvolvedores", onde é possível hospedar, armazenar e compartilhar projetos de desenvolvimento de software, facilitando a colaboração.
- GitLab: É uma plataforma completa de DevOps que permite armazenar projetos, gerenciar o histórico de alterações (usando o Git), automatizar o ciclo de desenvolvimento de software e colaborar em projetos de maneira integrada.

O que é Git e para que ele serve ?

Em um ambiente de equipe, o acesso ao código de outros colaboradores é apenas o ponto de partida. É fundamental ir além, mantendo um registro detalhado do histórico de arquivos e das modificações realizadas. Essa prática se torna crucial quando trabalhamos em grupo, onde diversas alterações podem ser consolidadas em um único movimento. No contexto do Git, essa consolidação é conhecida como commit, que, em tradução literal, representa um "compromisso" ou "confirmação" das alterações em um repositório.

Como o Git funciona ?

Mas, afinal, como o Git guarda todas as informações? Todos os nossos arquivos, junto com seus respectivos históricos, são armazenados em um repositório. No passado, diversos sistemas gerenciavam esses repositórios, como o CVS (Sistema de Versões Concorrentes) e o SVN (Subversion do Apache).

O Git surge como uma alternativa com um modelo de funcionamento ainda mais interessante: ele é um sistema de controle de versão distribuído. Isso significa que cada pessoa colaboradora possui uma cópia completa do repositório, e não apenas o "servidor principal". Uma das grandes vantagens desse modelo é a liberdade de desenvolver offline, permitindo que cada um realize seus commits e outras operações sem a necessidade de uma conexão constante com o servidor.

Em resumo, o Git é um sistema de controle de versão distribuído, uma ferramenta essencial que registra e gerencia o histórico de alterações de forma colaborativa.

BOLSA FUTURO DIGITAL
SOFTEX PERNAMBUCO
Softex
MINISTÉRIO DA CIÊNCIA, TECNOLOGIA E INOVAÇÃO
GOVERNO FEDERAL BRASIL UNIÃO E RECONSTRUÇÃO

Controle de versão

Um Sistema de Controle de Versão, ou VCS (Version Control System), monitora o histórico de alterações à medida que pessoas e equipes colaboram em projetos. Através dele, os desenvolvedores podem revisar o histórico do projeto para responder a perguntas fundamentais como:

- Quais alterações foram feitas?
- Quem as fez?
- Quando foram realizadas?
- Por que foram necessárias?

Por meio de branches, os desenvolvedores podem propor e testar alterações no código de produção de forma isolada e segura.

BOLSA FUTURO DIGITAL
SOFTEX PERNAMBUCO
Softex
MINISTÉRIO DA CIÊNCIA, TECNOLOGIA E INOVAÇÃO
GOVERNO FEDERAL BRASIL UNIÃO E RECONSTRUÇÃO

Repositório

Um repositório é um diretório que guarda os arquivos de um projeto e, fundamentalmente, todo o histórico de suas alterações.

Tipos de Repositórios

- Repositório Local: Fica armazenado no seu computador. Ele é criado com o comando `git init` e permite salvar alterações e criar um histórico do projeto localmente.
- Repositório Remoto: Fica armazenado em uma plataforma na nuvem, como GitHub ou GitLab. É o ponto central de colaboração da equipe e se conecta ao repositório local para sincronizar as alterações (com os comandos `git push` e `git pull`).

Commit

Um *commit* é como um instantâneo do seu projeto. Ele salva todas as alterações realizadas nos arquivos, criando um ponto de referência no histórico do repositório.
O que um commit faz:

- Registra alterações: Ele encapsula um conjunto específico de modificações no código ou em outros arquivos do projeto.
- Fornece contexto: Inclui uma mensagem descritiva para explicar o propósito das alterações, o que é fundamental para o rastreamento do histórico.
- Permite rastreamento: Cada *commit* facilita a navegação pela evolução do projeto, tornando mais fácil entender as mudanças e reverter para versões anteriores, se necessário.

Características de um Commit:

- Atômico: Cada *commit* deve representar uma única mudança lógica, como "corrigir um bug de login" ou "adicionar uma nova funcionalidade".
- Imutável: Uma vez que o *commit* é feito, ele não pode ser alterado diretamente. No entanto, há formas avançadas (como `git rebase`) de reescrever o histórico.
- Identificável: Cada *commit* possui um identificador único e exclusivo (*hash* SHA-1) que permite que ele seja rastreado com precisão no histórico do projeto.

Branch

Uma *branch* (ou ramo) é uma linha de desenvolvimento independente dentro de um repositório Git. Ela permite que você trabalhe em novas funcionalidades, correções de bugs ou outras alterações sem interferir na linha principal do projeto (geralmente chamada de `main` ou `master`).

Utilidade das Branches:

- Trabalho Paralelo: Permite trabalhar em diferentes tarefas ao mesmo tempo de forma isolada.
- Separação de Versões: Ajuda a separar versões estáveis do código de versões experimentais.
- Colaboração em Equipe: Facilita a colaboração, pois cada desenvolvedor pode ter sua própria linha de trabalho.

Processo de Mesclagem (*Merge*):

Quando o trabalho em uma *branch* é concluído e aprovado, ele pode ser mesclado (*merge*) de volta à *branch* principal, incorporando as alterações ao projeto.

Merge

O *merge* (mesclagem) é o processo de integrar as alterações feitas em uma *branch* de volta para outra, geralmente a *branch* principal (`main` ou `master`). Quando você executa um *merge*, o Git compara as modificações de ambas as *branches* e as combina em um novo ponto no histórico. Se as alterações não entrarem em conflito, a mesclagem é feita automaticamente. Caso contrário, o Git solicita que você resolva os conflitos manualmente antes de finalizar a operação.

Objetivo do Merge:

- Consolidar o Desenvolvimento: Combina funcionalidades ou correções de diferentes *branches* em uma única linha de desenvolvimento.
- Facilitar a Colaboração: Permite que várias pessoas trabalhem em partes distintas do código sem sobrescrever o trabalho umas das outras, integrando o resultado de forma organizada e controlada.

Pull e Push

Esses comandos permitem que você envie ou receba alterações, sincronizando o seu repositório local com o repositório remoto.

Git Pull

O comando `git pull` é usado para atualizar o repositório local com as alterações feitas no repositório remoto. Ele é, na verdade, uma combinação de dois comandos: ele primeiro "baixa" (`fetch`) as alterações mais recentes e, em seguida, as "mescla" (`merge`) à sua *branch* atual.

- Quando usar: Sempre que você quiser obter as mudanças feitas por outros desenvolvedores ou as alterações realizadas diretamente no repositório remoto, garantindo que seu repositório local esteja sincronizado.

Git Push

O comando `git push` é usado para enviar as alterações do seu repositório local para o repositório remoto. Após fazer seus *commits* localmente, você usa o push para compartilhar essas alterações com outros desenvolvedores e atualizar a versão remota do projeto.

- Quando usar: Sempre que você concluir uma funcionalidade, correção ou outra alteração importante e desejar que essas mudanças sejam refletidas no repositório remoto, tornando-as acessíveis a outros colaboradores.

Git Rebase e Git log

Git Rebase

O comando git rebase aplica as alterações de uma *branch* em cima de outra de forma linear. Ele reescreve o histórico de *commits* para criar uma linha de desenvolvimento mais limpa, evitando os *commits* de mesclagem que o git merge pode criar.

Ao contrário do merge, que combina as duas *branches* em um novo *commit*, o rebase move os *commits* da sua *branch* para o topo dos *commits* da *branch* base, resultando em um histórico mais fácil de ler e seguir.

Git Log

O comando `git log` exibe o histórico de *commits* de um repositório, apresentando uma lista detalhada de todas as alterações. Para cada *commit*, ele mostra informações essenciais como: o identificador (*hash*), o autor, a data e a mensagem do *commit*. Por padrão, os *commits* são listados do mais recente para o mais antigo. Você pode navegar pelo histórico com as setas do teclado e pressionar q para sair da visualização.

Git Fetch

Git fetch – Baixa as alterações do repositório remoto sem integrá-las O que faz: O comando git fetch baixa todas as atualizações do repositório remoto (novos commits, branches, tags), mas não faz automaticamente a integração dessas alterações no seu repositório local. Ou seja, ele traz as mudanças para o seu repositório local, mas não as aplica nas branches que você está trabalhando

Dicas

Como faço pra utilizar o código de outro Desenvolvedor?

Para usar o código de outro desenvolvedor, você pode clonar o repositório ou fazer um pull para pegar as alterações mais recentes.
Se ainda não tem o repositório localmente, use: git clone [URL do repositório] Se já tem o repositório local, execute o comando: git pull origin [nome da branch] Isso traz as alterações mais recentes da branch do repositório remoto para o seu repositório local.

Github

O GitHub é uma das plataformas de hospedagem de código-fonte mais populares, utilizando o Git como seu sistema de controle de versão. Ele é uma ferramenta essencial que permite a desenvolvedores colaborar em projetos de software, armazenar seu código em repositórios remotos e compartilhar seu trabalho com a comunidade.

Principais Funcionalidades:

- GitHub Actions: Uma ferramenta de automação integrada que permite criar fluxos de trabalho personalizados para tarefas como integração contínua (CI), entrega contínua (CD) e testes automatizados.
- Documentação e Wikis: Oferece espaços para criar e manter documentação detalhada para os projetos, facilitando o entendimento e a colaboração.
- GitHub Pages: Permite hospedar sites estáticos diretamente do repositório, ideal para documentação de projetos, blogs ou portfólios.
- Controle de Acesso e Permissões: Oferece ferramentas robustas para gerenciar quem pode visualizar, modificar ou contribuir para um repositório, garantindo a segurança do projeto.
- Comunidade e Projetos Open-Source: Atua como um hub central para a colaboração global, onde milhões de projetos de código aberto são desenvolvidos e mantidos.

GitLab

O GitLab é uma plataforma Git e DevOps completa, com a proposta de ajudar desenvolvedores a monitorar, testar e implantar seu código. Embora tenha começado como um repositório Git de código aberto e auto-hospedado, a plataforma evoluiu significativamente. Hoje, o GitLab se posiciona como uma solução unificada para todas as etapas do ciclo de desenvolvimento, oferecendo uma ampla gama de recursos integrados.

Principais Características do GitLab:

- CI/CD Integrado e Nativo: O principal diferencial do GitLab é o seu sistema de CI/CD (Integração Contínua e Entrega Contínua) que já vem integrado e pronto para usar, permitindo automatizar o fluxo de trabalho desde o teste até o *deploy*.
- Ferramenta de DevOps Completa: O GitLab se posiciona como uma plataforma "all-in-one", cobrindo todas as etapas do ciclo de DevOps, que inclui: Planejar, Criar, Verificar, Empacotar, Lançar, Configurar, Monitorar e Garantir a Segurança.
- Opção de Auto-Hospedagem (*Self-Hosted*): Além da versão na nuvem (SaaS), o GitLab mantém a opção de auto-hospedagem, um recurso importante para empresas que precisam manter seus dados e código em seus próprios servidores.
- Foco em Segurança Integrada: O GitLab oferece recursos de segurança que escaneiam o código, as dependências e os containers em busca de vulnerabilidades, tudo dentro do mesmo fluxo de trabalho.

Github vs Gitlab

GitLab e GitHub são plataformas populares de hospedagem de repositórios Git. Embora ofereçam recursos semelhantes, existem diferenças significativas que podem influenciar a escolha dos desenvolvedores.
O GitHub se destaca por sua vasta e ativa comunidade, sendo a escolha preferida para projetos *open-source*, e por oferecer uma grande variedade de integrações com ferramentas externas. Já o GitLab se diferencia por seus recursos mais completos e integrados para o planejamento e gerenciamento de todo o ciclo de vida do desenvolvimento (SDLC). Por fim, é importante notar que o GitLab e o GitHub não pertencem à mesma empresa: o GitLab é de propriedade da GitLab Inc., enquanto o GitHub é uma subsidiária da Microsoft.

BOLSA FUTURO DIGITAL
SOFTEX PERNAMBUCO
Softex
MINISTÉRIO DA CIÊNCIA, TECNOLOGIA E INOVAÇÃO
GOVERNO FEDERAL BRASIL UNIÃO E RECONSTRUÇÃO

Jogando e aprendendo

A plataforma Learn Git Branching é uma ferramenta interativa e visual projetada para ensinar o Git, com um foco especial nos conceitos de branches (ramos) e merges (mesclagem).
Para que ela serve?
Ela serve para ajudar iniciantes e desenvolvedores a:

- Entender o Git visualmente: Ao contrário de um tutorial de texto, a plataforma representa o histórico do Git como um grafo. Quando você digita um comando como `git commit` ou `git merge`, você vê instantaneamente a árvore de branches se alterando na tela.
- Aprender na prática: A plataforma é organizada em "níveis" e desafios. Para passar de um nível, você precisa digitar os comandos corretos para manipular o histórico do Git e alcançar um objetivo específico.
- Experimentar sem riscos: Você pode praticar e testar comandos sem o medo de bagunçar um projeto real. A plataforma funciona como um "sandbox", onde você pode cometer erros e aprender com eles.

Link: https://learngitbranching.js.org/?locale=pt_BR

SOFTEX PERNAMBUCO

Softex

MINISTÉRIO DA CIÊNCIA, TECNOLOGIA E INOVAÇÃO

GOVERNO FEDERAL BRASIL UNIÃO E RECONSTRUÇÃO